

BAB VII

PREDICTOR – MODEL PREDIKSI GNN UNTUK TUGAS KHUSUS

Bab ini menjelaskan mengenai model-model prediktor GNN yang akan digunakan dalam penelitian seperti GCN, GAT, GIN, dan GraphSAGE. Penjelasan yang akan ada dalam bab ini mencakup tugas klasifikasi bagaimana model dioptimalkan untuk memprediksi atribut graf. Selain penjelasan mengenai model prediktor, bab ini juga akan membahas hasil uji coba model prediktor.

7.1 Model Prediksi GNN pada GraphXAI

GraphXAI menyediakan berbagai model prediksi berbasis Graph Neural Network (GNN) yang dirancang untuk mengolah data graf dan menghasilkan prediksi untuk berbagai tugas, seperti klasifikasi node dan klasifikasi graf. Model-model prediksi ini mencakup Graph Convolutional Networks (GCN), Graph Attention Networks (GAT), Graph Isomorphism Networks (GIN), dan GraphSAGE. Setiap model ini memiliki keunikan dalam cara mereka mengagregasi dan memproses informasi dari node dan tetangganya untuk menghasilkan prediksi yang akurat. GraphXAI memungkinkan peneliti untuk menggunakan model-model ini dengan cara yang lebih transparan dan dapat dijelaskan, sehingga memudahkan pemahaman tentang bagaimana setiap model membuat keputusan berdasarkan struktur graf yang kompleks.

Dengan menggunakan GraphXAI, peneliti dapat mengakses dan mengeksplorasi bagaimana model-model prediksi GNN ini mengolah informasi pada berbagai level. Sebagai contoh, GCN mengandalkan teknik konvolusi yang memperbarui representasi node dengan cara menggabungkan informasi dari tetangga-tetangga terdekat. Dalam GraphXAI, peneliti dapat melihat bagaimana informasi topologi dan fitur node dari tetangga digunakan untuk menghasilkan representasi yang lebih baik, serta bagaimana model ini memprediksi label untuk

node yang belum terlabel dengan memperhitungkan informasi graf secara keseluruhan.

Selain itu, GAT menggunakan mekanisme attention untuk memberikan bobot yang berbeda pada setiap tetangga berdasarkan relevansi mereka terhadap prediksi yang dibuat. GraphXAI memungkinkan untuk memeriksa bagaimana model ini memprioritaskan tetangga tertentu dalam proses pembaruan representasi, yang memberikan wawasan mengenai keputusan model yang lebih dinamis dan kontekstual. Di sisi lain, GIN dirancang untuk memiliki daya diskriminasi yang lebih kuat dalam membedakan struktur graf yang sangat mirip, yang memungkinkan model ini untuk secara efektif menangani masalah klasifikasi graf yang lebih kompleks.

GraphSAGE memiliki pendekatan yang berbeda dengan menggunakan teknik induktif untuk menghasilkan representasi node dengan menyampling tetangga dalam graf yang besar. GraphXAI mendukung pemahaman bagaimana GraphSAGE memilih sampel tetangga dan bagaimana informasi dari node-node yang terpilih tersebut digabungkan menggunakan fungsi agregasi yang berbeda, seperti mean, LSTM, atau pooling. Model ini sangat berguna untuk graf yang besar dan dinamis, di mana informasi hanya tersedia sebagian dan dapat berubah seiring waktu.

Melalui GraphXAI, peneliti dapat memahami secara lebih mendalam cara kerja dan kelebihan masing-masing model GNN dalam memproses data graf. Dengan kemampuan untuk menguji dan menganalisis keputusan yang diambil oleh model-model ini, GraphXAI membantu peneliti memastikan bahwa model yang digunakan tidak hanya memberikan hasil yang akurat, tetapi juga dapat dijelaskan dan dipertanggungjawabkan, terutama dalam aplikasi-aplikasi penting yang mengandalkan keputusan berbasis graf.

7.2 Graph Convolutional Network

Graph Convolutional Networks (GCN) adalah model Graph Neural Network (GNN) yang pertama kali diperkenalkan oleh Kipf & Welling pada tahun 2017. Model ini dirancang untuk mengolah data yang memiliki struktur graf, di

mana node (simpul) dihubungkan oleh sisi (edge) dan informasi penting tersebar di seluruh struktur graf tersebut. GCN bekerja dengan cara menggabungkan informasi dari tetangga sebuah node untuk memperbarui representasi node tersebut. Proses ini dilakukan secara berulang pada beberapa lapisan, di mana pada setiap lapisan, node akan menggabungkan informasi dari tetangganya dan memperbarui representasi dirinya. Hal ini memungkinkan GCN untuk belajar representasi yang lebih baik dengan memanfaatkan hubungan antar node dalam graf.

Secara sederhana, GCN dapat dijelaskan seperti ini: setiap node di graf memulai dengan fitur tertentu, dan di setiap lapisan GCN, fitur tersebut diperbarui dengan menggabungkan informasi dari node-node tetangga. Proses ini memungkinkan model untuk menangkap pola-pola dan informasi struktural yang ada dalam graf. Model ini sangat efektif digunakan dalam tugas seperti node classification, di mana peneliti ingin memprediksi label atau kategori untuk setiap node di dalam graf.

Di dalam GraphXAI, GCN digunakan untuk analisis dan prediksi berbasis graf dengan memvisualisasikan bagaimana agregasi informasi dari tetangga dilakukan pada setiap lapisan. GraphXAI memungkinkan kita untuk memahami bagaimana informasi topologi graf dan fitur dari tetangga node digabungkan untuk memperbarui representasi node. Fitur ini sangat penting karena memungkinkan peneliti untuk melihat bagaimana model belajar dari data graf dan bagaimana keputusan prediksi dibuat.

Rumus utama yang digunakan dalam representasi node single layer GCN adalah sebagai berikut

$$H = f((\tilde{D}^{-\frac{1}{2}}(\tilde{A} + I)\tilde{D}^{-\frac{1}{2}})XW) \dots\dots\dots(7.1)$$

Dimana:

- H adalah representasi node setelah satu lapisan GCN.
- W adalah parameter model.
- A adalah matriks adjacency (representasi hubungan antar node).

- $\tilde{A} = A + I$ adalah matriks adjacency yang ditambah dengan identitas untuk menangani self-loops pada graf.
- D adalah matriks derajat
- X adalah fitur node.
- f adalah fungsi aktivasi, seperti ReLU.

GCN digunakan dalam banyak aplikasi berbasis graf, salah satunya adalah node classification. Misalnya, dalam graf jejaring sosial, setiap node bisa mewakili pengguna, dan sisi bisa mewakili hubungan antar pengguna. GCN dapat digunakan untuk memprediksi kategori atau atribut tertentu dari pengguna yang belum diketahui berdasarkan informasi dari pengguna lain yang terhubung dalam graf. Contoh lainnya adalah jaringan sitasi akademik, di mana GCN dapat digunakan untuk mengklasifikasikan artikel-artikel penelitian ke dalam topik-topik tertentu berdasarkan hubungan sitasi yang ada dalam graf.

Tabel 7.1
Kelebihan dan Kekurangan GCN

Kelebihan	Kekurangan
Sederhana dan efisien. GCN memiliki arsitektur yang relatif sederhana, namun sangat efektif dalam mengolah data graf.	GCN bersifat transduktif, artinya model ini memerlukan seluruh graf saat pelatihan, dan tidak dapat langsung diterapkan pada graf baru yang tidak terlihat selama pelatihan. Ini membuat GCN kurang cocok untuk situasi di mana graf terus berkembang dan data baru terus masuk.
Efektif untuk tugas semi-supervised. GCN bekerja dengan sangat baik dalam situasi di mana hanya sebagian node yang dilabeli, dan sisanya dapat diprediksi menggunakan informasi dari tetangga mereka.	

Dengan GraphXAI, peneliti dapat lebih memahami bagaimana GCN bekerja dengan cara yang lebih transparan. Visualisasi yang disediakan oleh GraphXAI membantu peneliti untuk memeriksa kontribusi tetangga node dalam pembaruan representasi dan memastikan bahwa model ini bekerja sebagaimana

mestinya, terutama dalam aplikasi-aplikasi yang memerlukan kejelasan dan keterpercayaan tinggi pada keputusan yang diambil oleh model.

7.3 Graph Attention Network

Graph Attention Networks (GAT), yang diperkenalkan oleh Velickovic et al. pada tahun 2018, adalah sebuah model Graph Neural Network (GNN) yang memanfaatkan mekanisme perhatian (attention) untuk memperbaiki cara informasi digabungkan antara node-node yang ada dalam sebuah graf. Berbeda dengan GCN (Graph Convolutional Networks) yang mengandalkan agregasi rata-rata informasi dari tetangga, GAT memberikan bobot perhatian yang berbeda pada setiap tetangga, memungkinkan model untuk memberikan perhatian yang lebih besar pada node-node yang lebih relevan dalam konteks prediksi yang sedang dilakukan.

Mekanisme perhatian ini dilakukan menggunakan self-attention, yang menghitung bobot perhatian untuk setiap pasangan node yang terhubung. Prosesnya dimulai dengan menghitung skor perhatian e_{ij} yang menunjukkan seberapa penting fitur node j bagi node i . Kemudian, skor perhatian ini dinormalisasi menggunakan fungsi softmax, menghasilkan koefisien perhatian a_{ij} . Koefisien ini digunakan untuk menggabungkan informasi dari tetangga dengan bobot yang telah ditentukan oleh perhatian tersebut GraphSAGE.

Di GraphXAI, model GAT dapat dianalisis dengan visualisasi bobot perhatian yang dihitung untuk setiap tetangga. Ini memberikan wawasan mengenai bagaimana GAT memutuskan node mana yang harus lebih diperhatikan dalam setiap langkah agregasi. Melalui GraphXAI, peneliti dapat memeriksa koefisien perhatian ini dan mempelajari bagaimana model memprioritaskan informasi yang datang dari tetangga tertentu. Hal ini sangat berguna untuk memahami mengapa model memilih untuk memberi bobot lebih pada beberapa node dalam graf dan bagaimana keputusan tersebut mempengaruhi prediksi model.

Koefisien perhatian antara node i dan node j dihitung dengan rumus berikut:

$$e_{ij} = \text{LeakyReLU}(a^T [Wh_i \parallel Wh_j]) \dots\dots\dots (7.2)$$

Dimana:

- h_i adalah representasi fitur dari node i pada lapisan sebelumnya (atau input fitur pada lapisan pertama).
- h_j adalah representasi fitur dari tetangga node i , yaitu node j .
- W adalah matriks bobot yang dipelajari oleh model, digunakan untuk mengubah (mentransformasi) fitur dari node i dan node j .
- α adalah vektor parameter perhatian yang dipelajari selama pelatihan untuk menghitung bobot perhatian.
- $\parallel$ untuk menyatakan operasi concatenation (penggabungan) antara representasi node i dan node j setelah ditransformasi dengan matriks bobot W .
- LeakyReLU adalah fungsi aktivasi non-linear yang digunakan untuk memperkenalkan nonlinearitas dalam model dan untuk menghindari masalah "vanishing gradient". Fungsi LeakyReLU adalah sebagai berikut:

$$\text{LeakyReLU}(x) = \max(\alpha x, x) \dots\dots\dots(7.3)$$

dengan α adalah konstanta kecil (misalnya, 0.01) yang memungkinkan gradien kecil saat input negatif.

Setelah skor perhatian dihitung, nilai tersebut perlu dinormalisasi agar bobot perhatian antara node i dan semua tetangganya ($j \in N(i)$) menjadi nilai yang proporsional dan jumlahnya sama dengan 1. Bobot perhatian yang dinormalisasi ini dihitung menggunakan softmax, yang memastikan bahwa bobot perhatian yang lebih besar diberikan pada tetangga yang lebih relevan. Rumusnya adalah sebagai berikut:

$$a_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in N_i} \exp(e_{ik})} \dots\dots\dots(7.4)$$

Penjelasan Elemen-elemen Rumus:

- e_{ij} adalah skor perhatian antara node i dan node j , yang dihitung sebelumnya.

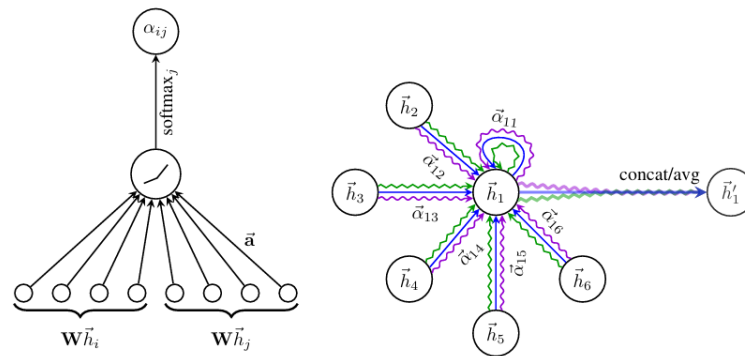
- $\exp(e_{ij})$ merupakan fungsi eksponensial diterapkan pada skor perhatian e_{ij} untuk memastikan nilai bobot perhatian positif.
- N_i adalah tetangga dari node i . Untuk setiap node i , akan dihitung skor perhatian terhadap semua tetangganya yaitu $j \in N(i)$
- Sum di bawah adalah penjumlahan eksponensial dari skor perhatian untuk semua tetangga node i ($j \in N(i)$) digunakan untuk menormalkan skor perhatian. Hal ini untuk memastikan bahwa jumlah total bobot perhatian untuk semua tetangga node i adalah 1.

Setelah menghitung bobot perhatian antara node i dan tetangganya, langkah berikutnya adalah memperbarui representasi node i dengan menggabungkan informasi dari tetangganya, yang diberi bobot sesuai dengan skor perhatian yang telah dihitung. Representasi dari setiap tetangga h_j dikalikan dengan bobot perhatian a_{ij} yang relevan untuk node i , lalu dijumlahkan untuk menghasilkan representasi node yang diperbarui h'_i . Proses ini dapat dijelaskan dengan rumus berikut:

$$h'_i = \sigma\left(\sum_{j \in N(i)} a_{ij} W h_j\right) \dots\dots\dots(7.5)$$

Penjelasan Elemen-elemen Rumus:

- h'_i adalah representasi node i yang diperbarui setelah menggabungkan informasi dari tetangga.
- a_{ij} adalah bobot perhatian yang dihitung untuk setiap tetangga j .
- h_j adalah representasi fitur dari tetangga node j .
- W adalah matriks bobot yang digunakan untuk mengubah fitur dari tetangga.
- σ adalah fungsi aktivasi seperti ReLU yang digunakan untuk memperkenalkan nonlinearitas pada pembaruan representasi node.



Gambar 7.1
Mekanisme Kerja Graph Attention Networks¹

Gambar 7.1 menjelaskan secara visual bagaimana mekanisme perhatian (attention mechanism) dalam Graph Attention Networks (GAT) bekerja, dengan dua bagian yaitu kiri dan kanan. Pada gambar kiri, ditunjukkan bagaimana perhatian antara node pusat i dan tetangganya j dihitung. Mula-mula, fitur dari kedua node ini masing-masing ditransformasikan menggunakan matriks bobot pembelajaran W , menghasilkan representasi $W\vec{h}_j$ dan $W\vec{h}_i$. Kedua vektor ini kemudian digabungkan (concatenate) menjadi satu vektor panjang, yang selanjutnya diproses oleh vektor bobot perhatian $\vec{a}$ melalui operasi dot product. Hasilnya kemudian dilewatkan ke dalam fungsi aktivasi LeakyReLU untuk menambahkan non-linearitas. Skor yang dihasilkan disebut e_{ij} , yaitu skor perhatian mentah. Skor ini kemudian dinormalisasi menggunakan fungsi softmax, menghasilkan bobot perhatian α_{ij} , yang menunjukkan seberapa besar perhatian node i terhadap tetangganya j . Proses ini diulang untuk semua tetangga node i , sehingga node i dapat memberikan bobot berbeda kepada tiap tetangganya berdasarkan relevansi informasi mereka.

Sementara itu, gambar 7.1 bagian kanan mengilustrasikan bagaimana proses ini berlangsung secara keseluruhan untuk satu node (node 1) dalam konteks multi-head attention. Dalam ilustrasi tersebut, node 1 dikelilingi oleh enam tetangga (node 2 hingga node 6), dan dari node 1 mengarah beberapa panah berwarna-warni ke

¹ Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. Graph Attention Networks, hlm 4.

masing-masing tetangga. Warna dan gaya panah yang berbeda menandakan bahwa perhatian dihitung oleh tiga attention head yang berbeda secara paralel. Setiap head menghitung bobot perhatian α_{ij} secara independen terhadap semua tetangga, menghasilkan agregasi informasi yang unik pada setiap head. Setelah semua head melakukan agregasi fitur dari tetangga, hasil dari ketiganya digabungkan untuk menghasilkan representasi akhir node i yang diperbarui, yaitu h'_i . Mekanisme multi-head ini memperkuat kemampuan model untuk menangkap berbagai sudut pandang hubungan antar node, serta meningkatkan stabilitas pembelajaran.

GAT sangat efektif digunakan dalam tugas-tugas yang melibatkan node classification dan graph classification, terutama ketika graf memiliki tetangga yang sangat berbeda relevansinya. Misalnya, dalam graf jejaring sosial, GAT dapat digunakan untuk memprediksi minat atau kategori pengguna, di mana beberapa tetangga pengguna mungkin lebih relevan daripada yang lain dalam mempengaruhi prediksi. Selain itu, GAT juga dapat digunakan dalam inductive learning, di mana model harus dapat membuat prediksi pada data yang tidak terlihat selama pelatihan, yang membuatnya sangat berguna untuk graf yang terus berkembang.

Tabel 7.2
Kelebihan dan Kekurangan GAT

Kelebihan	Kekurangan
Fleksibel dalam memberikan perhatian. GAT memungkinkan bobot perhatian yang berbeda pada setiap tetangga, yang memberikan fleksibilitas lebih dalam menangani graf yang heterogen.	Biaya komputasi yang lebih tinggi. Proses perhitungan perhatian untuk setiap tetangga bisa mahal secara komputasi, terutama jika graf memiliki banyak node.
GAT dapat digunakan pada graf yang tidak terlihat selama pelatihan, karena inductive learning.	

7.4 Graph Isomorphism Network

Graph Isomorphism Network (GIN) adalah salah satu model Graph Neural Network (GNN) yang dirancang secara khusus untuk memiliki kekuatan representasional maksimum dalam membedakan struktur graph. Model ini diperkenalkan oleh Xu et al. pada tahun 2019 dengan tujuan utama mengatasi kelemahan model-model GNN sebelumnya, seperti GCN dan GraphSAGE, yang dalam beberapa kasus tidak mampu membedakan graph dengan struktur berbeda namun memiliki agregasi fitur tetangga yang identik. GIN mendasarkan arsitekturnya pada prinsip dari Weisfeiler-Lehman (WL) test, sebuah metode klasik untuk menguji isomorfisme dua graph, dan mengadopsinya dalam bentuk operasi agregasi berbasis penjumlahan dan fungsi transformasi non-linear menggunakan Multi-Layer Perceptron (MLP). GIN terbukti secara teori memiliki kemampuan representasi yang setara dengan WL test, menjadikannya sangat andal dalam membedakan struktur graph yang kompleks dan mirip.

Berbeda dari GCN atau GAT yang menggunakan rata-rata atau perhatian dalam agregasi, GIN menggunakan penjumlahan murni (sum aggregator) dari fitur node tetangga. Setelah menjumlahkan semua fitur dari tetangga node v , GIN menambahkan nilai dari node itu sendiri yang telah dikalikan dengan faktor $(1 + \epsilon)$ adalah skalar kecil yang dapat diatur secara manual atau dilatih bersama parameter lainnya. Hasil agregasi tersebut kemudian dilewatkan ke dalam MLP yang akan menghasilkan representasi node yang baru. Dengan pendekatan ini, GIN memastikan bahwa struktur tetangga yang berbeda akan menghasilkan output yang berbeda pula, meningkatkan kemampuan model dalam membedakan graf. Di dalam GraphXAI, model GIN digunakan untuk menganalisis dan menjelaskan klasifikasi graph berdasarkan struktur yang kompleks. Fitur interpretabilitas dari GraphXAI memungkinkan pengguna untuk mengamati bagaimana perubahan kecil dalam struktur atau fitur graf dapat memengaruhi representasi dan prediksi model GIN. Karena GIN peka terhadap perbedaan struktur, GraphXAI sangat membantu dalam menunjukkan peran tiap node dan koneksi dalam hasil akhir klasifikasi, serta memvisualisasikan konstelasi fitur yang membuat dua graph dianggap berbeda oleh

model. Ini sangat penting dalam bidang-bidang seperti kimia, di mana struktur molekul sangat menentukan klasifikasinya.

Formulasi utama dalam GIN untuk pembaruan representasi node adalah sebagai berikut:

$$h_v^{(k)} = \text{MLP}^{(k)} \left((1 + \epsilon^{(k)}) \cdot h_v^{(k-1)} + \sum_{u \in N(v)} h_u^{(k-1)} \right) \dots\dots\dots (7.6)$$

Penjelasan komponen:

- $h_v^{(k)}$ adalah representasi node v pada lapisan ke- k ,
- $h_v^{(k-1)}$ adalah representasi sebelumnya (lapisan ke- $k - 1$),
- $N(v)$ adalah himpunan tetangga dari node v ,
- $\epsilon^{(k)}$ adalah skalar yang dapat dipelajari (untuk model GIN- ϵ) atau diset tetap (misalnya 0 untuk GIN-0),
- MLP adalah fungsi transformasi non-linier neural network multilapis. MLP ini mengambil agregasi dari tetangga (misalnya, penjumlahan atau distribusi) dan memprosesnya untuk menghasilkan representasi yang lebih kompleks dan tidak linier.

GIN sangat efektif digunakan dalam tugas klasifikasi graf (*graph classification*), di mana tujuan utama adalah memetakan keseluruhan struktur graf ke dalam satu label atau kategori yang merepresentasikan sifat atau fungsi dari graf tersebut. Contohnya termasuk klasifikasi molekul berdasarkan sifat kimia seperti mutagenisitas, toksisitas, atau efektivitas biologisnya dalam pengobatan. Selain itu, GIN juga banyak diterapkan dalam analisis jaringan protein untuk menentukan apakah suatu struktur berfungsi sebagai enzim atau bukan, serta untuk mengidentifikasi bagian-bagian penting atau substruktur yang berkontribusi terhadap fungsi spesifik dari graf. Kemampuan GIN yang luar biasa ini diperoleh dari arsitekturnya yang menggunakan Multi-Layer Perceptron (MLP) sebagai fungsi agregasi, alih-alih hanya menggunakan operasi sederhana seperti rata-rata (*mean*) atau maksimum (*max*) seperti pada GNN lainnya. MLP memungkinkan model untuk belajar fungsi non-linear yang kompleks dalam menggabungkan

informasi dari tetangga setiap node, sehingga memberikan fleksibilitas dan kekuatan representasi yang lebih tinggi.

Tabel 7.3
Kelebihan dan Kekurangan GIN

Kelebihan	Kekurangan
GIN terbukti secara teori dan eksperimen mampu membedakan lebih banyak jenis struktur graph dibandingkan GCN dan GraphSAGE.	Karena menggunakan MLP yang ekspresif, GIN bisa terlalu menyesuaikan data latih jika tidak dilengkapi regularisasi yang baik (rentan overfitting)
Meskipun sangat kuat, GIN memiliki arsitektur yang relatif sederhana (penjumlahan + MLP).	GIN lebih cocok untuk graph-level prediction daripada node-level prediction seperti pada GCN atau GAT.
	Kurang efisien pada graf besar karena operasi sum pada seluruh tetangga dapat menjadi berat.

7.5 Graph Sample and Aggregation

GraphSAGE (Graph Sample and Aggregation) adalah model Graph Neural Network (GNN) yang dirancang untuk pembelajaran induktif, yang berarti model ini dapat melakukan prediksi pada data baru yang tidak terlihat selama pelatihan. Ini sangat berguna untuk aplikasi graf yang terus berkembang, seperti jejaring sosial, di mana node baru dapat muncul kapan saja setelah model dilatih. Keunggulan utama dari GraphSAGE adalah kemampuannya untuk mengagregasi informasi dari tetangga dalam graf, tetapi dengan cara yang lebih efisien dan dapat diskalakan. Alih-alih mengolah seluruh graf sekaligus, GraphSAGE mengambil sampel tetangga untuk setiap node dan menggunakan teknik agregasi untuk menghasilkan representasi node yang lebih kompak.

Proses agregasi dilakukan dalam beberapa lapisan, di mana setiap lapisan menggabungkan informasi dari tetangga node (hingga kedalaman tertentu). GraphSAGE menggunakan beberapa jenis aggregator untuk menggabungkan informasi dari tetangga node, tergantung pada jenis data dan tugas yang dihadapi. Tiga jenis aggregator utama yang digunakan adalah mean aggregator, pooling aggregator, dan LSTM aggregator. Mean aggregator adalah yang paling sederhana,

di mana representasi node baru dihitung dengan mengambil rata-rata dari fitur tetangga, cocok untuk graf di mana informasi dari tetangga dianggap memiliki relevansi yang serupa. Pooling aggregator menggunakan operasi pooling (seperti max pooling) untuk memilih nilai tertinggi (atau terendah) dari tetangga, memberi perhatian lebih pada tetangga yang paling relevan. Sementara itu, LSTM aggregator menggunakan Long Short-Term Memory (LSTM) untuk mengagregasi informasi dari tetangga dengan mempertimbangkan urutan dan konteksnya, cocok untuk graf yang lebih dinamis dan kompleks. Masing-masing aggregator ini memungkinkan GraphSAGE untuk menangani graf dengan cara yang fleksibel dan efisien, tergantung pada sifat dan kompleksitas data graf yang digunakan.

Di dalam GraphXAI, GraphSAGE digunakan untuk menghasilkan representasi node yang dapat dipahami dan diinterpretasikan. GraphXAI memungkinkan visualisasi bagaimana informasi dari tetangga node digabungkan untuk menghasilkan representasi node yang lebih baik. Dengan menggunakan teknik multi-hop dalam GraphSAGE, informasi yang lebih luas tentang graf dapat diperoleh meskipun hanya berdasarkan tetangga langsung, yang membantu dalam memprediksi label node yang belum terlihat atau mengklasifikasikan graf besar yang dinamis. GraphXAI memberikan kemudahan untuk menganalisis dan mengevaluasi kontribusi dari berbagai tetangga terhadap pembaruan representasi node, yang sangat penting dalam tugas-tugas klasifikasi node.

Formula utama untuk pembaruan representasi node pada GraphSAGE adalah sebagai berikut:

$$h_v^{(k)} = \sigma(W^{(k)} \cdot AGGREGATE(\{h_u^{(k-1)} \mid u \in N(v)\}) + b^{(k)}) \dots\dots(7.7)$$

Di sini:

- $h_v^{(k)}$ adalah representasi node v pada lapisan ke- k ,
- $h_u^{(k-1)}$ adalah representasi node tetangga u pada lapisan sebelumnya (ke- $k - 1$),
- $N(v)$ adalah himpunan tetangga dari node v ,

- AGGREGATE adalah fungsi agregator yang digunakan untuk menggabungkan fitur dari tetangga,
- $W^{(k)}$ dan $b^{(k)}$ adalah matriks bobot dan bias pada lapisan ke- k ,
- Σ adalah fungsi aktivasi ReLU.

GraphSAGE sangat efektif untuk pembelajaran induktif, yang berarti model dapat digunakan untuk menghasilkan representasi dari node-node yang tidak ada dalam graf selama pelatihan. Misalnya, dalam jejaring sosial, GraphSAGE dapat digunakan untuk memprediksi atribut atau preferensi pengguna baru berdasarkan interaksi mereka dengan pengguna yang sudah ada.

Tabel 7.4
Kelebihan dan Kekurangan GraphSAGE

Kelebihan	Kekurangan
Dapat digunakan untuk node baru yang tidak ada dalam data pelatihan, sangat berguna untuk graf yang dinamis dan terus berkembang.	Pilihan agregator yang tepat sangat penting untuk kinerja GraphSAGE, dan model ini memerlukan eksperimen untuk menentukan fungsi agregator terbaik sesuai dengan dataset.
Dengan menggunakan teknik sampling untuk memilih tetangga, GraphSAGE lebih efisien dalam memproses graf besar dibandingkan dengan metode transduktif seperti GCN.	Penggunaan sampling untuk mengurangi beban komputasi bisa menyebabkan hilangnya beberapa informasi penting jika sampling tidak dilakukan dengan hati-hati.
Dapat menggunakan berbagai jenis agregator (mean, pooling, LSTM) yang memungkinkan model untuk menangkap informasi yang lebih beragam dan sesuai dengan karakteristik graf yang berbeda.	

7.6 Jenis Klasifikasi pada Predictor

Pada subbab ini, akan membahas dua jenis tugas klasifikasi yang umum digunakan dalam Graph Neural Networks (GNN), yaitu Node Classification dan Graph Classification. Meskipun kedua tugas ini berfokus pada klasifikasi dalam graf, keduanya memiliki tujuan yang berbeda, pendekatan yang berbeda, dan

aplikasi yang beragam. Subbab ini akan memberikan gambaran mendalam mengenai bagaimana masing-masing tugas ini diterapkan menggunakan model GNN, serta bagaimana mereka berbeda dalam proses dan hasil akhir.

7.6.1 Klasifikasi Node

Klasifikasi Node adalah tugas di mana model GNN digunakan untuk memprediksi label atau kelas dari setiap node dalam graf. Setiap node dalam graf, seperti individu dalam jaringan sosial atau atom dalam molekul, memiliki beberapa fitur atau atribut. Dalam tugas ini, model belajar untuk memprediksi label untuk setiap node berdasarkan informasi yang ada pada node tersebut serta informasi yang berasal dari tetangga-tetangga node tersebut.

Sebagai contoh, dalam jaringan sosial, sebuah node bisa mewakili pengguna, dan model GNN bertugas untuk memprediksi kategori pengguna tersebut, seperti apakah mereka bagian dari komunitas tertentu. GNN mengagregasi informasi dari pengguna-pengguna lain yang terhubung dalam jaringan untuk menghasilkan prediksi yang lebih akurat.

Tugas ini menggunakan berbagai model GNN, seperti GCN, GAT, dan GIN, yang memiliki kemampuan untuk mengagregasi informasi dari node-node tetangga. Model ini bekerja dengan mengubah fitur-fitur node menjadi representasi yang lebih kaya, yang kemudian digunakan untuk memprediksi kelas dari setiap node dalam graf.

7.6.2 Proses dan Teknik dalam Klasifikasi Node

Pada Node Classification, model GNN menggabungkan informasi dari node dan tetangga node untuk memprediksi label. Proses utamanya melibatkan beberapa langkah:

1. Agregasi Fitur Node

Langkah pertama dalam Node Classification adalah agregasi fitur dari setiap node dan tetangganya. Setiap node dalam graf memiliki atribut atau fitur, yang bisa berupa vektor numerik yang menggambarkan karakteristik atau informasi penting tentang node tersebut, seperti usia pengguna di jaringan sosial, atau ukuran atom

dalam graf molekul. Model GNN mulai dengan mengambil fitur dari node itu sendiri, serta fitur dari node-node tetangga yang terhubung padanya melalui edges. Informasi ini kemudian digabungkan untuk membentuk representasi yang lebih kaya dari node tersebut. Fitur ini biasanya disusun dalam matriks x , di mana setiap baris mewakili fitur dari satu node. Proses agregasi ini penting untuk memberikan konteks pada setiap node berdasarkan interaksinya dengan node-node lain dalam graf.

2. Propagation

Setelah fitur node dikumpulkan, informasi dari node-node tetangga disebarkan atau dipropagasi melalui edges yang menghubungkan mereka. Proses ini disebut propagation. Tujuan utama dari propagation adalah untuk memungkinkan setiap node untuk menggabungkan informasi dari tetangga-tetangganya, yang membantu memperkaya representasi node tersebut. Misalnya, dalam jaringan sosial, informasi dari pengguna lain yang terhubung langsung (tetangga) akan mempengaruhi klasifikasi pengguna utama. Dalam proses ini, informasi yang lebih global mengenai hubungan antar node akan dihimpun untuk menciptakan representasi yang lebih informatif, yang dapat digunakan untuk memprediksi label dari node tersebut.

3. Lapisan Konvolusi Graf

Setiap lapisan dalam GNN melakukan konvolusi graf yang secara efektif melakukan agregasi informasi dari node tetangga. Konvolusi graf ini adalah inti dari cara GNN bekerja. Pada setiap lapisan konvolusi, informasi dari node-node tetangga disebarkan ke node yang sedang diproses. Hal ini memperbarui representasi node tersebut dengan memasukkan informasi yang relevan dari tetangga-tetangga terdekatnya. Setiap lapisan konvolusi menghasilkan representasi yang lebih kaya dan lebih kompleks untuk setiap node, berdasarkan hubungan yang ada di graf. Proses ini diulang untuk beberapa lapisan, yang memungkinkan informasi untuk merambat lebih jauh di seluruh graf, memungkinkan setiap node untuk belajar dari informasi yang lebih luas.

4. Aktivasi Non-linear

Setelah setiap lapisan konvolusi, model GNN menerapkan fungsi aktivasi non-linear seperti ReLU (Rectified Linear Unit). Fungsi aktivasi ini berfungsi untuk menambah non-linearitas pada model, yang sangat penting untuk membantu model memahami pola yang lebih kompleks dalam data. Tanpa fungsi aktivasi non-linear, model hanya akan belajar representasi linier dari data, yang membatasi kemampuannya untuk menangkap pola yang lebih rumit. Fungsi ReLU mengubah nilai negatif menjadi nol dan mempertahankan nilai positif, yang mempercepat proses pelatihan dan membantu dalam mengatasi masalah vanishing gradient.

5. Prediksi Label Node

Setelah beberapa lapisan konvolusi dan aktivasi, model akhirnya menghasilkan representasi akhir dari setiap node. Representasi ini mencakup informasi yang telah digabungkan dari node dan tetangga-tetangganya, yang menggambarkan node tersebut dalam konteks yang lebih luas. Representasi ini kemudian digunakan untuk memprediksi label kelas node tersebut. Prediksi dilakukan dengan menggunakan lapisan output (biasanya berupa lapisan linear atau fully connected layer) yang memetakan representasi akhir dari node ke dalam jumlah kelas yang diinginkan. Hasil prediksi ini adalah label atau kategori dari node, yang bisa berupa klasifikasi seperti jenis komunitas pengguna dalam jaringan sosial atau jenis sel dalam analisis data biologis.

6. Loss Function

Untuk melatih model GNN, perlu untuk mengevaluasi seberapa baik model memprediksi label node. Di sinilah loss function berperan. Salah satu loss function yang umum digunakan dalam Node Classification adalah cross-entropy loss, yang mengukur seberapa besar kesalahan antara label yang diprediksi dan label yang benar (ground truth). Cross-entropy loss menghitung perbedaan antara distribusi probabilitas yang diprediksi oleh model dan distribusi probabilitas yang sebenarnya dari label kelas. Semakin kecil nilai loss, semakin baik model dalam memprediksi label yang benar. Loss ini digunakan untuk backpropagation, di mana gradient dihitung dan digunakan untuk memperbarui parameter model (misalnya, bobot

dalam lapisan konvolusi) melalui optimisasi (biasanya dengan algoritma seperti Adam).

7.6.3 Klasifikasi Graf

Sementara Node Classification berfokus pada klasifikasi setiap node dalam graf, Graph Classification bertujuan untuk memprediksi label atau kelas dari seluruh graf. Dalam tugas ini, model GNN digunakan untuk menghasilkan representasi dari seluruh graf dengan mengagregasi informasi dari semua node dan hubungan antar node. Setelah itu, representasi graf tersebut digunakan untuk memprediksi kelas dari graf secara keseluruhan.

Contohnya, graf dapat digunakan untuk mewakili molekul, dan model GNN bertugas untuk memprediksi jenis molekul tersebut, seperti apakah molekul itu organik atau anorganik. Dalam hal ini, setiap graf mewakili satu molekul, dan tugas klasifikasi bertujuan untuk menentukan kategori atau sifat molekul tersebut. Model-model GNN yang digunakan untuk Graph Classification sering kali melibatkan teknik global pooling, seperti global mean pooling, yang mengagregasi informasi dari seluruh node dalam graf untuk menghasilkan representasi tunggal graf tersebut. Dengan representasi graf ini, model dapat melakukan prediksi kelas graf secara keseluruhan.

7.6.4 Proses dan Teknik dalam Klasifikasi Graf

Pada Graph Classification, proses utamanya adalah mengklasifikasikan seluruh graf ke dalam satu label atau kelas berdasarkan informasi yang ada di dalam graf tersebut. Proses ini melibatkan beberapa tahap yang berbeda dari Node Classification, karena di sini kita tidak hanya memprediksi label untuk setiap node, tetapi untuk seluruh graf. Berikut adalah tahapan utama dalam proses Graph Classification:

1. Pengolahan Node

Proses pertama dalam Graph Classification mirip dengan Node Classification, yaitu pengolahan fitur node. Model Graph Neural Networks (GNN) memulai dengan memproses fitur dari setiap node dalam graf. Setiap node dalam

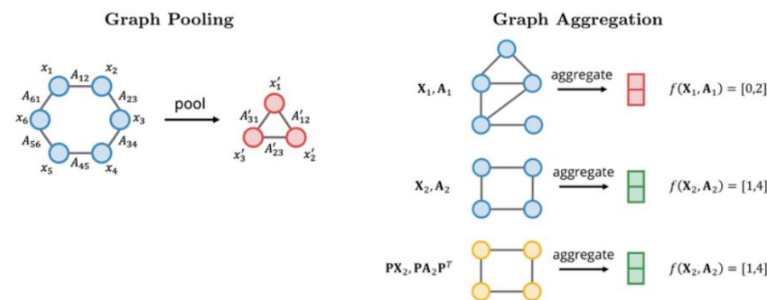
graf memiliki fitur yang merepresentasikan informasi atau karakteristik tertentu. Misalnya, dalam konteks kimia, setiap node dalam graf bisa mewakili sebuah atom dalam molekul, dan fitur node bisa menggambarkan sifat atom tersebut, seperti ukuran atom atau elektronegativitas. Begitu juga dalam jaringan sosial, fitur node bisa mencakup informasi tentang individu dalam graf.

Setelah fitur node diambil, model GNN mengagregasi informasi dari node tersebut dan dari tetangga-tetangganya. Teknik propagasi digunakan untuk menyebarkan informasi antar node yang terhubung, dan ini membantu node untuk "memahami" konteks yang lebih luas dari tetangganya dalam graf.

2. Agregasi Informasi Node

Setelah informasi dari setiap node diproses dan direpresentasikan melalui beberapa lapisan konvolusi, langkah berikutnya dalam Graph Classification adalah menggabungkan informasi dari seluruh node dalam graf menjadi satu representasi tunggal. Proses ini dikenal sebagai global pooling, dan merupakan tahap kritis yang menentukan seberapa baik karakteristik keseluruhan graf dapat ditangkap dalam satu vektor representasi. Teknik yang paling umum digunakan adalah mean pooling (atau sering disebut global mean pooling), di mana vektor fitur dari setiap node dihitung rata-ratanya sehingga menghasilkan satu vektor tunggal yang merangkum seluruh informasi graf.

Selain mean pooling, terdapat beberapa alternatif teknik agregasi lainnya yang dapat digunakan bergantung pada karakteristik data dan tujuan klasifikasi. Max pooling mengambil nilai maksimum dari setiap dimensi fitur di seluruh node, yang berguna untuk menangkap fitur yang paling menonjol dalam graf. Sum pooling menjumlahkan semua vektor fitur node, yang dapat mempertahankan informasi tentang ukuran graf dan intensitas keseluruhan fitur. Teknik yang lebih canggih seperti attention-based pooling menggunakan mekanisme perhatian untuk memberikan bobot yang berbeda pada setiap node berdasarkan relevansinya terhadap tugas klasifikasi, sementara hierarchical pooling melakukan agregasi secara bertahap melalui beberapa tingkat untuk mempertahankan informasi struktural yang lebih kompleks.



Gambar 7.2
Perbedaan antara Graph Pooling dan Graph Aggregation²

Gambar 7.2 menjelaskan perbedaan mendasar antara graph pooling dan graph aggregation. Di sisi kiri, dapat dilihat contoh graph pooling, yaitu proses di mana struktur graf secara eksplisit diubah misalnya, dengan menggabungkan beberapa node menjadi supernode. Proses ini menghasilkan graf baru yang lebih sederhana dan padat.

Di sisi kanan, kita melihat graph aggregation, yaitu proses di mana fitur dari node dan tetangganya digabungkan untuk memperbarui representasi node, tanpa mengubah struktur graf. Teknik ini umum digunakan dalam lapisan-lapisan konvolusi GNN seperti pada GCN, GAT, atau GIN. Fungsi agregasi yang digunakan bisa berupa mean, sum, atau max, dan proses ini bersifat lokal, hanya melibatkan node dan tetangganya. Global mean pooling pada Graph Classification merupakan bagian dari teknik graph pooling, karena bertujuan menggabungkan representasi dari seluruh node dalam graf menjadi satu vektor global yang mewakili graf secara keseluruhan.

3. Prediksi Kelas Graf

Setelah global pooling menghasilkan representasi tunggal dari graf, tahap berikutnya adalah prediksi kelas untuk graf tersebut. Representasi tunggal yang dihasilkan oleh pooling berfungsi sebagai input untuk lapisan linear atau fully connected layer, yang akan memprosesnya lebih lanjut untuk menghasilkan

² Graph pooling and graph aggregation (https://www.researchgate.net/figure/Graph-pooling-and-graph-aggregation-Graph-pooling-left-accepts-a-graph-signal-and_fig4_343441194 diakses pada 18 Mei 2025)

prediksi kelas. Lapisan linear ini memetakan representasi vektor dari graf ke dalam ruang kelas yang diinginkan, seperti kelas molekul kimia atau jenis jaringan sosial. Di sini, representasi akhir dari graf (setelah pooling) dipetakan ke dalam ruang kelas menggunakan lapisan linear, yang memberikan output berupa probabilitas untuk setiap kelas yang mungkin. Hasil akhirnya adalah label kelas yang paling mungkin untuk graf tersebut, yang menunjukkan kategori atau sifat yang dimiliki oleh graf tersebut. Biasanya, softmax digunakan pada output akhir untuk menghasilkan distribusi probabilitas atas kelas-kelas yang ada.

7.7 Contoh Implementasi dan Penggunaan Predictor

Pada implementasi ini, digunakan dataset sintetis ShapeGGen untuk melakukan Node Classification menggunakan berbagai arsitektur Graph Neural Network (GNN) dari pustaka GraphXAI, seperti GCN, GIN, GAT, dan GraphSAGE. Prosesnya dimulai dari penyiapan data, inisialisasi model, pelatihan model, hingga evaluasi akhir yang akan dievaluasi menggunakan performance metrics pada bab 6 dan visualizer pada bab 8. Kode implementasi subbab ini dapat dilihat pada Lampiran Segmen Program A.13.

Pertama, dilakukan import terhadap berbagai pustaka dan fungsi yang diperlukan. Library numpy dan torch digunakan untuk komputasi numerik dan manipulasi tensor. Dari sklearn.utils, fungsi `compute_class_weight` digunakan untuk menghitung bobot kelas berdasarkan distribusi label, berguna ketika data memiliki jumlah instance yang tidak seimbang antar kelas. Selain itu, dari `graphxai.gnn_models.node_classification.testing`, mengimpor keempat arsitektur model GNN yang akan diuji, yaitu `GCN_3layer_basic`, `GIN_3layer_basic`, `GAT_3layer_basic`, dan `GSAGE_3layer`. Fungsi `train` dan `test` digunakan untuk proses pelatihan dan evaluasi model (baris 1-11).

Selanjutnya, data diambil dari objek dataset menggunakan fungsi `get_graph(use_fixed_split=True)`. Parameter `use_fixed_split=True` memastikan data sudah dipisah menjadi bagian pelatihan dan pengujian secara konsisten (baris 14). Perangkat komputasi kemudian ditentukan melalui `torch.device("cuda" if torch.cuda.is_available() else "cpu")`, yang akan menggunakan GPU jika tersedia,

atau CPU sebagai alternatif (baris 15). Lalu, jumlah kelas dalam label target dihitung dari atribut `data.y` menggunakan `torch.unique(data.y)`, yang dibutuhkan saat inisialisasi model (baris 17).

Setelah itu, pengguna dapat menentukan model yang ingin digunakan melalui variabel `model_choice`. Dalam kode ini, nilai awalnya adalah 'GCN', namun dapat diganti ke 'GIN', 'GAT', atau 'GSAGE' sesuai kebutuhan. Berdasarkan pilihan tersebut, model yang sesuai akan diinisialisasi menggunakan konfigurasi standar `hidden_channels=32`, `input_feat=dataset.n_features` untuk menyesuaikan jumlah fitur input pada node, dan `classes=num_classes` untuk menyesuaikan jumlah kelas output. Model kemudian dipindahkan ke perangkat (`.to(device)`) agar dapat dijalankan dengan efisien di GPU jika tersedia (baris 23-39).

Setelah model siap, langkah berikutnya adalah mengatur optimizer dan fungsi loss. Optimizer yang digunakan adalah Adam, dengan learning rate 0.001 dan `weight_decay=0.001` untuk regularisasi (baris 42). Selanjutnya, dilakukan penyesuaian bobot kelas menggunakan `compute_class_weight` dari Scikit-Learn. Fungsi ini menghitung bobot seimbang berdasarkan jumlah kemunculan tiap kelas, agar model tidak terlalu bias terhadap kelas mayoritas. Bobot ini dikonversi ke tensor PyTorch dan dipindahkan ke perangkat (device) (baris 46-48). Bobot tersebut kemudian dimasukkan ke dalam `CrossEntropyLoss`, yang merupakan fungsi loss yang umum untuk klasifikasi multi-kelas (baris 51).

Proses pelatihan model dilakukan dalam loop sebanyak 300 epoch. Pada setiap epoch, fungsi `train()` dijalankan untuk melakukan satu iterasi pelatihan. Fungsi ini mengatur proses forward pass (prediksi), menghitung loss menggunakan criterion, melakukan backpropagation, dan memperbarui bobot model menggunakan optimizer. Seluruh parameter model diperbarui pada setiap epoch untuk meminimalkan loss (baris 54-55).

Setelah pelatihan selesai, performa model diuji menggunakan fungsi `test()` (baris 58). Fungsi ini menghitung berbagai metrik evaluasi, yaitu:

- F1-score: metrik yang menggabungkan precision dan recall, berguna saat distribusi kelas tidak seimbang.
- Accuracy: proporsi prediksi yang benar dari seluruh data uji.

- Precision dan Recall: menunjukkan seberapa tepat dan lengkap prediksi model terhadap kelas positif.
- AUROC (Area Under Receiver Operating Characteristic): mengukur kemampuan model dalam membedakan antara kelas.
- AUPRC (Area Under Precision-Recall Curve): mengukur keseimbangan antara precision dan recall dalam berbagai ambang prediksi.

Terakhir, nilai F1-score dan AUROC dicetak sebagai metrik utama evaluasi (baris 61-62). Ini memberikan gambaran awal seberapa baik model melakukan klasifikasi node pada graf sintetis dari ShapeGGen. Hasil evaluasi menggunakan model GCN 3 Layer menunjukkan bahwa model mencapai F1-score sebesar 0.9091 dan AUROC sebesar 1.0000. Nilai-nilai ini menunjukkan bahwa model memiliki performa yang sangat baik dalam mengklasifikasikan label setiap node pada graf yang diuji.

Nilai F1-score sebesar 0.9091 berarti bahwa model berhasil mengklasifikasikan mayoritas node secara benar, baik dari segi keakuratan maupun kelengkapan (recall). Sementara itu, AUROC (Area Under the Receiver Operating Characteristic Curve) sebesar 1.0000 menunjukkan bahwa model memiliki kemampuan yang sempurna dalam membedakan antara kelas yang ada. Nilai ini menyiratkan bahwa model tidak melakukan kesalahan dalam membedakan node dari kelas berbeda. Nilai evaluasi seperti ini menunjukkan bahwa model GNN yang digunakan sangat efektif dalam memahami struktur dan informasi fitur dari node dan tetangganya, serta mampu memanfaatkan informasi tersebut untuk menghasilkan prediksi yang sangat akurat. Namun, meskipun nilai AUROC sempurna, perlu dilakukan evaluasi lanjutan terhadap generalisasi model pada dataset lain atau pada graf dengan struktur yang lebih kompleks, guna memastikan model tidak hanya overfit terhadap pola tertentu pada ShapeGGen.